

HW9: loop优化

- 姓名：陈雨萱
- 学号：23300240034
- 专业：计算机科学与技术（拔尖人才计划）

关键技术实现

本次任务主要是对中间代码做两件与循环相关的事：用支配关系找回边和自然循环，包括循环头与循环体，以及在假定已有 preheader 的前提下，把循环不变、且无访存/调用副作用的运算外提到循环外。程序读入 .4-ssa-withflow-xml.quad，打印循环信息，并输出优化后的 .4-ssa-loopopt.quad。

findloopheader.cc

该文件的核心目标，是在控制流图CFG上识别循环，并把每个循环对应的“循环头”以及循环体中的基本块整理出来，供后续循环优化使用。整体采用的是“回边+ 自然循环”方法。整个文件实际上只做了两件事：识别 CFG 中哪些边是回边，据回边恢复对应的自然循环。

1. `naturalLoopForBackEdge` 函数用于在已知控制流图中存在回边 $m \rightarrow n$ 的情况下，恢复该回边对应的自然循环结构，其中 n 为循环头， m 为循环尾。函数首先将循环头 n 初始化加入循环集合，然后以 m 作为起点，通过栈结构进行反向遍历。具体来说，算法不断从栈中取出节点，并检查该节点是否已经被加入循环集合，如果尚未加入，则将其加入，并进一步访问其所有前驱节点。由于 CFG 中的前驱关系表示控制流的反向传播路径，因此该过程等价于在反向图上进行深度优先搜索，从而逐步扩展所有能够回到循环头的节点。最终，该函数能够收集到所有属于该自然循环的基本块集合，并返回完整的循环体。
2. `findLoopHeaders` 函数的主要作用是在函数级控制流图上识别所有循环结构，并构建“循环头到循环体”的映射关系。函数首先通过 `loopOptNoteNewFlowProgram` 标记当前分析流程的开始，并初始化 `LoopHeaderMap` 作为结果容器。当控制流信息 `FuncFlowInfo` 或其 CFG 信息为空时，函数直接返回空的循环结构映射。随后，函数遍历 CFG 中的所有控制流边，对于每一条边 $u \rightarrow v$ ，结合支配关系信息判断是否构成回边，即检查 v 是否支配 u 。若满足支配条件，则认为该边为回边，并调用 `naturalLoopForBackEdge` 恢复对应的自然循环体。恢复出的循环集合会被归并到以 v 为循环头的映射结构中，从而支持多个回边指向同一循环头的情况。最后，函数将每个循环头及其对应的循环体封装为 `LoopHeader` 对象集合，并写入 `LoopHeaderMap` 中返回，完成整个函数级循环结构分析。
3. `loopOptNoteNewFlowProgram` 与 `loopOptAfterFindLoopHeaders` 未在当前文件中实现，只是做了声明，真正定义在 [loophoistfunc.cc](#)，属于循环优化框架中的阶段控制函数。前者用于标记一个新的函数控制流图分析开始。后者则在循环头识别完成后被调用，用于标记循环分析阶段结束。

它是一个基于 Quad IR 的编译器循环优化模块，主要是在完构建 CFG 控制流图辅助结构、识别并排序循环、对多函数的 IR 输出顺序进行控制以及执行循环不变量的外提，保证最终 IR 输出顺序稳定一致。具体的函数如下：

1. `loopOptNoteNewFlowProgram` 函数用于标记当前循环优化分析所属的“程序流上下文”，其主要作用是识别是否进入一个新的编译程序批次。函数通过读取 `FuncFlowInfo` 中的 `programLastLabelNum` 与 `programLastTempNum` 作为当前程序的唯一标识，如果该标识与全局缓存 `s_flowProgKey` 不一致，则说明当前处理的是新的程序输入，此时函数会重置循环分析相关的全局状态，包括循环调用计数器 `s_findLoopCallsThisProgram` 以及函数访问顺序缓存 `s_hoistVisitOrder`。该机制保证在多程序或多函数批处理场景下，循环优化状态不会跨程序污染。
2. `loopOptAfterFindLoopHeaders` 函数用于记录当前程序中已经完成了一次 `findLoopHeaders` 调用。每调用一次该函数，全局计数器 `s_findLoopCallsThisProgram` 加一，用于统计当前程序涉及循环分析的函数数量。该计数器后续用于判断是否已经收集完整整个程序所有函数的循环信息，从而触发后续多函数 IR 重排序操作。
3. `sourceEmitRankForName` 函数用于为函数名分配输出优先级，以控制最终 IR 或汇编输出顺序。其中函数名包含 **\$main** 的函数被视为入口函数，因此返回优先级 0；其他普通函数返回 1。该排序策略保证主函数始终优先输出，从而符合程序执行逻辑与调试习惯。
4. `reorderFuncsToSourceEmitOrder` 函数用于对多个 `QuadFuncDecl` 的 IR 结构进行统一重排序，使其输出顺序符合预期的源程序 `emit` 顺序。函数首先对所有函数结构进行快照保存，包括基本块列表、函数名、参数、临时变量编号等信息，避免排序过程中破坏原数据。随后通过排序索引对函数进行排序，排序规则为：优先根据 `sourceEmitRankForName` 区分主函数与普通函数，其次按照函数名字典序排序。最后将排序结果写回原函数对象，从而实现 IR 级别的函数重排。
5. `maybeFixMultiFuncProgramOrder` 该函数用于在多函数程序中触发最终的函数顺序修正逻辑。每处理一个函数时，会将其加入全局访问序列 `s_hoistVisitOrder`。当当前已收集函数数量达到本程序循环分析调用次数 `s_findLoopCallsThisProgram` 且函数数量大于等于 2 时，说明所有函数均已完成循环优化分析，此时调用 `reorderFuncsToSourceEmitOrder` 对函数顺序进行统一调整，并清空缓存。该机制保证多函数程序的 IR 输出顺序稳定且一致。
6. `buildCfgFromFunc` 函数用于从 Quad IR 构建控制流图（CFG），生成基本块之间的前驱和后继关系。函数遍历每个 `QuadBlock`，以其入口标签作为节点编号，并根据其出口标签集合建立 `successor` 关系。随后再通过 `successor` 反向构建 `predecessor` 关系，从而形成完整 CFG 结构。该函数为后续支配分析、循环检测和优化变换提供基础图结构。
7. `buildLabelToBlock` 函数用于建立“标签 → 基本块”的映射关系，使得后续可以通过 `block label` 快速定位对应的 `QuadBlock`。函数遍历函数中的所有基本块，并将其 `entry_label` 映射到对应 `block`。该结构用于快速访问 CFG 节点，是 `loop hoisting` 与 IR 修改的重要索引结构。
8. `buildDefiningBlocks` 函数用于构建“临时变量定义位置分析表”，即记录每个临时变量（Temp）在哪个基本块中被定义。函数遍历所有基本块中的语句，并检查语句的 `def` 集合，将变量编号映射到其定义所在的 `block label`。该信息用于后续判断循环不变量（`loop invariant`），是 `loop hoisting` 的关键数据依赖。

- 9. isHoistablePureStmt函数用于判断一条四元语句是否可以参与循环外提。只有满足“纯计算类型”的语句才可能被 hoist，包括 MOVE、MOVE_BINOP 和 PTR_CALC 三类操作。这些语句不会产生副作用，因此可以安全移动。
- 10. usesOnlyInvariantTemps函数用于判断一条语句是否只依赖循环不变量。它检查语句的 use 集合中的所有变量是否都属于 invariant 集合。如果全部满足，则说明该语句在循环中不会随迭代变化，可以被外提。
- 11. findPreheaderLabel函数用于寻找循环头对应的 preheader。preheader 的定义是：所有指向 loop header 的前驱中，不属于 loop body 的那个基本块。函数从 predecessor 集合中筛选出所有不在循环体内的节点，并返回其中最小 label 作为稳定选择结果。如果不存在这样的节点，则返回 -1。
- 12. findPreheaderInsertIndex函数用于确定在 preheader block 中插入 hoisted 代码的位置。它从后向前扫描基本块语句列表，并跳过结尾的控制流终止语句（如 JUMP、CJUMP、RETURN），确保 hoisted 代码插入在正确的执行语义位置之前，从而不会破坏控制流结构。
- 13. nestedIn + orderLoopsOutermostFirst这两个函数共同用于处理循环嵌套关系。nestedIn 判断一个循环是否嵌套在另一个循环内部
orderLoopsOutermostFirst 使用拓扑排序思想，根据嵌套关系对循环进行排序，确保外层循环先处理这样可以避免内层循环修改影响外层循环分析结果。
- 14. loopHoistFunc是整个文件最核心的优化函数，实现循环不变量外提。其整体流程为，首先函数检查输入合法性，并通过 RAII 结构 RegisterLoopHoistPass 确保多函数程序排序修正逻辑在函数退出时一定被触发。随后函数构建 CFG 相关结构，包括 predecessor/successor、label→block 映射以及变量定义块信息。

接着对所有循环按照“外层优先”的顺序进行遍历，对于每个循环先找到其 preheader，如果不存在则跳过。然后计算循环中的 invariant 变量集合，即所有在循环外定义的变量。基于该信息，函数在循环体内反复扫描语句，寻找满足以下条件的可外提语句：

- 语句类型是纯计算语句
- 语句只使用循环不变量
- 未被标记为已外提

在找到可外提语句后，将其加入 markedHoist 集合，并更新 invariant 集合（因为外提后可能产生新的不变量传播）。该过程循环进行直到不再有新语句可外提。

随后将所有可外提语句按照原始控制流顺序排序，并从原基本块中删除，再统一插入到 preheader 的合适位置（避开终止语句）。最后更新变量定义信息，以保证后续循环分析一致性。

开发过程和测试结果

开发过程如下：

Graph	Description	Date	Author	Commit
	添加了一些情况的判断，包括不可约流图、多循环外前驱、0 次进入循环以及多函数	17 May 2026 00:03	23300240034-chen...	87bcc613
	把main函数调整到最前面输出，符合参考答案的约定	16 May 2026 23:45	23300240034-chen...	cd8a9ea4
	feat:修改要求的两个文件和quadflow下的CMakeLists.txt文件名，基本实现loop优化	16 May 2026 23:22	23300240034-chen...	b41cf7c8
	Merge remote-tracking branch 'origin/master'	16 May 2026 23:18	23300240034-chen...	19858ef4
	完成LLVM的提交	16 May 2026 23:16	23300240034-chen...	af8e5090

make run 之后得到的所有测试的结果，与老师给出的参考测试结果完全一致。并且在终端得到的输出片段如下：

```
Block: 104 <- 102
Block: 107 <- 103 108
Block: 108 <- 107
Block: 109 <- 107
Block: 110 <-
Successors:
Block: 102 -> 103 104
Block: 103 -> 107
Block: 104 ->
Block: 107 -> 108 109
Block: 108 -> 107
Block: 109 -> 102
Block: 110 -> 102
Dominators:
Block: 102 <- 102 110
Block: 103 <- 102 103 110
Block: 104 <- 102 104 110
Block: 107 <- 102 103 107 110
Block: 108 <- 102 103 107 108 110
Block: 109 <- 102 103 107 109 110
Block: 110 <- 110
Immediate Dominators:
Block: 102 -> 110
Block: 103 -> 102
Block: 104 -> 102
Block: 107 -> 103
Block: 108 -> 107
Block: 109 -> 107
Block: 110 -> -1
Dominance Frontier:
Block: 102 -> 102
Block: 103 -> 102
Block: 104 ->
Block: 107 -> 102 107
Block: 108 -> 107
Block: 109 -> 102
Block: 110 ->
Dominator Tree:
Block: 102 -> 103 104
Block: 103 -> 107
Block: 104 ->
Block: 107 -> 108 109
Block: 108 ->
Block: 109 ->
Block: 110 -> 102
Loop headers for function __$main__^main: Header Label: 102, Body Blocks: {102 103 107 108 109 } Header Label: 107, Body Blocks: {107 108
Optimized function __$main__^main
Writing optimized Quad to file: opttest9.4-ssa-loopopt.quad
----Done---
```

参考资料

- 虎书中文版，“中间表示与基本块”，“数据流分析”，“循环优化”部分
- 大模型：DeepSeek/Cursor/ChatGPT，协助理解虎书内容和修改代码中遇到的对补充其他情况的考虑和是否tiao Zhengmain函数顺序的斟酌问题。